2 반복문

프로그래밍에서 반복문은 특정 명령문을 반복해서 실행해야 할 때 사용한다. 예를 들어 놀이공원 회전목마를 자동으로 20바퀴 운행하는 프로그램을 작성하려면 어떻게 해야 할까?

일반 명령 예

회전목마 1번째 바퀴 운행하시오.
회전목마 2번째 바퀴 운행하시오.
⋮
회전목마 20번째 바퀴 운행하시오.

반복문 사용 예

회전목마를 20바퀴 운행하시오.
범위 반복할 명령

```
01 # 프로그래밍으로 반복문 작성하기
02 for i in range(1, 21) :
03     print("회전목마", i, "번째 바퀴 운행하시오.")
```

▲ 실생활 속에서 반복문 활용 예

프로그래밍에서 1~100까지의 합을 구하는 과정을 살펴보면 다음과 같다.

▼ 반복문을 사용하지 않은 경우

```
01  t = 0   # t값을 0으로 초기화하기
02  t += 1
03  t += 2
 ⋮   ⋮
100 t += 99
101 t += 100
102 print(t)
```

t = t + 1로 쓸 수 있어요.

합을 구하는 명령문 100개를 나열해요.

▼ 반복문을 사용한 경우

```
01 t = 0
02 for a in range(1, 101) :
03     t += a   # 덧셈 100번 하기
04 print(t)
```

1부터 100까지의 정수를 생성하는 범위를 지정해요.

변수 a에 1부터 100까지의 숫자를 순차적으로 할당하여 특정 명령문을 반복해요.

변수 a가 1부터 100이 될 때까지 t += a 문장을 100번 수행해요.

이러한 반복문의 종류에는 for문, while문 등이 있다.

1 for문

주로 범위를 정해 놓고 특정 명령문을 반복하는 구조이다.

형식 1
for 변수 in 리스트[값, 값, …] :
반복 명령문

형식 2
for 변수 in 문자열 :
반복 명령문

형식 3
for 변수 in range(시작값, 종룟값, 증감값) :
반복 명령문
- 변수의 값은 '시작값'에서 '종룟값 –1'까지 '증감값'만큼 변화된다.
- 시작값이 0, 증감값이 1일 때는 시작값과 증감값을 생략할 수 있다.

예 for a in range(1, 6, 1) :
for a in range(1, 6) :
a값은 1부터 5까지 1씩 증가한다.

체크 주의하세요!
- 블록 처리를 위해 for문은 반복할 범위를 지정한 후 마지막에는 반드시 콜론(:)을 입력해야 한다.
- for문에서 반복할 명령문들은 같은 위치의 들여쓰기를 한다.

for문은 방법에 따라 단순 for문과 중첩 for문으로 나눌 수 있다.

(1) 단순 for문

한 개의 for문을 사용하는 구조로 범위에 리스트나 문자열, range( ) 함수를 사용할 수 있다.

예제 23 리스트에 저장된 값들을 하나씩 출력해 보자.

```
01 for a in [1, 2, 3] :
02   print(a)
```

실행 결과
```
1
2
3
```

리스트의 각 요소 [1, 2, 3]을 차례로 a 변수에 할당하고 a 변숫값을 출력해요.

하나 더 알기 range( ) 함수

데이터 범위를 한번에 지정할 수 있는 명령문으로 주로 for문과 함께 사용된다.

▶형식 range([시작값], 종룟값, [증감값])

- range( ) 함수 범위는 '시작값'에서 '종룟값 전의 값'까지 '증감값'만큼 변화된다.
- '시작값'이나 '증감값'은 생략 가능하며, 이 경우 '시작값'은 0, '증감값'은 1이다.

▼ 예제 23의 for문을 range( )로 표현하기

```
01 # 사용 예1: range(시작값, 종룟값, 증감값)
02 for i in range(0, 3, 1) :
03   print(i)
04 # 사용 예2: range(시작값, 종룟값)
05 for i in range(0, 3) :
06   print(i)
07 # 사용 예3: range(종룟값)
08 for i in range(3) :
09   print(i)
```

예제 24 문자를 하나씩 출력해 보자.

```
01 for a in "hello" :
02   print(a)
```

실행 결과

```
h
e
l
l
o
```

예제 25 일정 범위의 숫자를 출력해 보자.

```
01 for a in range(1, 4) :
02   print(a)
```

실행 결과

```
1
2
3
```

예제 26 스마트폰의 타이머 앱과 같이 초를 입력하여 0초가 될 때까지 줄어드는 타이머 프로그램을 만들어 보자.

```
01 import time
02 for i in range(5, 0, -1) :
03   print(i, "초 남았습니다.")
04   time.sleep(1)
05 print("종료")
```

실행 결과

```
5 초 남았습니다.
4 초 남았습니다.
3 초 남았습니다.
2 초 남았습니다.
1 초 남았습니다.
종료
```

참고 time.sleep()

파이썬에서 일정 시간 동작을 정지시킬 때 사용하는 명령문이다.

```
01 import time
02 # 1초 동안 일시적으로 정지하기
03 time.sleep(1)
```

예제 27 [실행 결과]를 참고하여 구구단의 단을 입력하면 해당 구구단을 출력하는 프로그램을 완성해 보자.

실행 결과

```
몇 단? 5 Enter↵
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
  ⋮
5 * 9 = 45
```

```
01 d = int(input("몇 단? "))
02 for i in range(1, [   10   ]) :
03   a = d * i
04   print( [   d, "*", i, "=", a   ] )
```

(2) 중첩 for문

for문 안에 또 다른 for문이 포함된 구조로 내부 for문이 끝나면 외부 for문을 실행한다. 중첩 for문의 수행 과정을 살펴보면 다음과 같다.

```
01  for i in range(2) :                # i를 0~1까지 반복하기
02    for j in range(3) :              # j를 0~2까지 반복하기
03      print("i = ", i, "j = ", j)    # i, j값 출력하기
```

실행 결과

외부 for 문	내부 for 문
i = 0 i = 0 i = 0	j = 0 j = 1 j = 2
i = 1 i = 1 i = 1	j = 0 j = 1 j = 2

외부 for문의 첫 번째 반복(i = 0) 실행 시 내부 for문의 반복 명령문을 3번 실행한다.

외부 for문의 두 번째 반복(i = 1) 실행 시 내부 for문의 반복 명령문을 3번 반복한다.

예제 28 구구단 2단부터 9단까지 출력해 보자.

```
01  for i in range(2, 10) :            # 외부 for문
02    for j in range(1, 10) :          # 내부 for문
03      print(i, "*", j, "=", i * j)
```

2단을 출력할 때는 2단이 외부 for문, 2단 안에서 1씩 증가하는 것은 내부 for문으로 수행해요.

실행 결과

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
  ⋮
9 * 7 = 63
9 * 8 = 72
9 * 9 = 81
```

예제 29 중첩 반복문을 사용하여 [문제 상황]에 맞는 모든 세트 메뉴 조합을 출력하는 프로그램을 작성해 보자.

문제 상황

- ○○놀이공원의 햄버거 가게는 팝콘과 음료를 조합하여 다양한 세트 메뉴를 제공하고 있다.
- 팝콘은 '기본', '카라멜', '어니언' 세 가지 종류가 있고, 음료는 '생수'와 '탄산음료' 두 가지 종류가 있다.

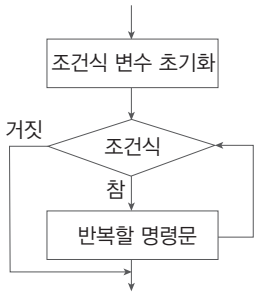
```
01 popcorn_options = ["기본", "카라멜", "어니언"]   # 팝콘과 음료의 옵션
02 drink_options = ["생수", "탄산음료"]
03 for popcorn in popcorn_options :                # 모든 세트 메뉴 조합 출력
04     for drink in drink_options :
05         print("세트 메뉴: ", popcorn, "팝콘,", drink)
```

실행 결과

```
세트 메뉴:  기본 팝콘, 생수
세트 메뉴:  기본 팝콘, 탄산음료
세트 메뉴:  카라멜 팝콘, 생수
세트 메뉴:  카라멜 팝콘, 탄산음료
세트 메뉴:  어니언 팝콘, 생수
세트 메뉴:  어니언 팝콘, 탄산음료
```

2 while문

조건식이 참인 동안만 내부의 명령문을 반복해서 실행하고 조건식이 거짓이면 반복문을 벗어난다.

형식	순서도	설명
변수 초기화 while 조건식 : 반복 명령문		• 조건식이 참인 동안 명령문을 반복 실행한다. • while문 내부에서 조건식의 변숫값을 증감하면서 명령문의 반복 횟수를 조절하기도 한다. • while문의 조건식에 사용되는 변수에 값을 미리 저장하는 경우가 많다.

체크 주의하세요!
- while문의 조건식에서 변수를 사용한다면 변수를 미리 초기화한다.
- while문 내부의 반복할 명령문은 같은 위치의 들여쓰기를 한다.

예제 30 while문으로 1부터 5까지 출력해 보자.

```
01  a = 1              # 조건식 변수 초기화
02  while a <= 5 :     # 조건식
03    print(a)         # a값 출력하기
04    a = a + 1        # a값을 1씩 증가하기
```

실행 결과
```
1
2
3
4
5
```

예제 30을 for문으로 표현해 볼까요?

```
01  for a in range(1, 6) :
02    print(a)
```

하나 더 알기 무한 반복

특정 명령을 무한히 반복하여 실행하는 것을 말한다. 무한 반복은 while문의 조건식 대신 True나 1을 쓰거나 조건식 결괏값을 항상 참이 되게 하여 만들 수 있다. 한편, 무한 반복을 벗어나기 위해서는 강제 종료를 하거나 while 내부에 break문을 사용할 수 있다.

```
01  while True :
02    print("!")
```

```
01  while 1 :
02    print("!")
```

▲ while문의 무한 반복 예

3 반복문의 제어

반복문에서 프로그램의 흐름을 제어하기 위한 명령문으로 break문, continue문 등이 있다.

(1) break문

반복문을 실행하다가 강제로 반복문을 벗어나야 할 경우에 사용한다.

예제 31 게임 프로그램을 실행하다가 q 또는 Q를 입력하면 게임을 종료하고 반복문을 빠져나가는 프로그램을 작성해 보자.

```
while 1 :                                   # 무한 반복
  n = input("프로그램을 벗어나려면 q 또는 Q를 입력하세요. ")
  if n == 'q' or n == 'Q' :                 # 'q' 또는 'Q'를 입력하면
     print("프로그램 나가기")                # '프로그램 나가기' 출력하기
     break                                  # while문 탈출하기
```

실행 결과

```
프로그램을 벗어나려면 q 또는 Q를 입력하세요. a Enter
프로그램을 벗어나려면 q 또는 Q를 입력하세요. b Enter
프로그램을 벗어나려면 q 또는 Q를 입력하세요. q Enter
프로그램 나가기
```

키보드로 q나 Q를 눌렀을 때 반복문을 벗어날 수 있어요.

(2) continue문

반복문을 실행하다가 특정 명령문을 건너뛰고 처음으로 돌아가서 반복문을 계속 실행할 경우에 사용한다.

예제 32 1부터 100 사이의 자연수 중 5의 배수 합을 출력해 보자.

```
total = 0                          # total 변수에 0으로 초기화하기
for i in range(1, 101) :           # i 변숫값을 1부터 100까지 반복하기
  if i % 5 != 0 :                  # i가 5의 배수가 아닌가?
     continue                      # 5의 배수가 아니라면 for문으로 돌아가기
  total += i                       # i가 5의 배수라면, total 변수에 i값 더하기
print(total)                       # 반복문이 종료되면 total값 출력하기
```

실행 결과

```
1050
```